



MaRDRe: User's Guide

Authors:
Roberto R. Expósito, Jorge Veiga, Jorge González-Domínguez
and Juan Touriño

January 9, 2017



Contents

1 Introduction **3**

2 Prerequisites **3**

3 Execution **4**
3.1 Command-line arguments 4
3.2 Configuration 4

4 Compilation **5**

5 Contact **6**

1 Introduction

MaRDRe [1] is a *de novo* MapReduce-based parallel tool to remove duplicated DNA reads through the clustering of single-end or paired-end sequences from FASTQ/FASTA datasets. Duplicate reads can be seen as identical or nearly identical sequences with some mismatches. Depending on the application scenario, duplicated or near-duplicated reads do not provide any interesting biological information but can increase memory requirements and computational time of downstream analysis. This tool allows researchers and bioinformatics to avoid the analysis of not necessary reads, reducing the time of subsequent procedures with the dataset (e.g., assemblies, mappings, etc.).

MaRDRe is the Big Data counterpart of ParDRe [2, 3], which employs HPC technologies (i.e., hybrid MPI/multithreading) to reduce runtime on multicore systems. Instead, MarDRe takes advantage of the MapReduce programming model originally developed by Google [4] to significantly improve ParDRe performance on distributed systems, specially on cloud-based infrastructures. Written in pure Java to maximize cross-platform compatibility, MaRDRe is built upon the open-source Apache Hadoop project [5], the most popular distributed computing framework for scalable Big Data processing.

This tool is distributed as free software and is publicly available at [1] under the GPLv3 license [6].

2 Prerequisites

In order to use MaRDRe, the prerequisites are:

1. Make sure you have Java Runtime Environment (JRE) version 1.7 or above
2. Make sure you have a working Hadoop distribution version 2.2 or above
   - HADOOP_HOME environmental variable must be set accordingly
3. Untar the downloaded MaRDRE distribution:
   - On Linux/Mac OS, just follow the instructions below:

```
[user@host ~]$ mkdir /path/to/mardre
[user@host ~]$ tar xzf MaRDRe-v1.0.tgz -C /path/to/mardre
```

   - Alternatively use your preferred archive extraction tool
4. Set MARDRE_HOME and PATH environmental variables:
   - On Linux/Mac OS you can set them in your profile or your shell configuration files (e.g., .bashrc). Follow the instructions below:

```
[user@host ~]$ export MARDRE_HOME=/path/to/mardre
[user@host ~]$ export PATH=$MARDRE_HOME/bin:$PATH
```

3 Execution

MarDRe can be executed by using the provided *mardrerun* command, which launches the MapReduce jobs to the Hadoop cluster. The only compulsory argument is the input dataset for single-end executions, or two datasets in case of paired-end executions. The input FASTQ/FASTA datasets must be previously stored on the Hadoop Distributed File System (HDFS) [7].

3.1 Command-line arguments

The available command-line arguments are:

- -i. Compulsory both in single-end and paired-end scenarios. String with the HDFS path to the sequence file in FASTA/FASTQ format.
- -p. Compulsory in paired-end scenarios. String with the HDFS path to the second sequence file in FASTA/FASTQ format.
- -m. Optional. Integer with the number of allowed mismatches to identify two reads as equivalent. The default value is 0.
- -l. Optional. Integer with the prefix length. During the map phase, MaRDRe emits as key the first l characters of each read. Then, the reads of the same key are compared in the reduce phase. Higher prefix-length usually leads to shorter computation but can miss some duplicates. The loss of accuracy is observed when removing near-duplicated reads. Reads in the same group have exactly the same prefix (i.e., mismatches are not allowed in the prefix). The longer the prefix, more near-duplicated can be missed. Lets use as example an scenario where we try to compress near-duplicated reads with up to one mismatch. If we use a prefix of length 20 to compare two reads that only have one mismatch in position 16, MaRDRe stores them in different groups, they are never compared, and both reads will be in the output. Otherwise, with a prefix of length 15 MaRDRe compares them and, as only one base is different, discards one of them. The default value is 20.

The following command removes the duplicates of identical reads (no mismatches) of a single-end dataset using a prefix length of 15 and 8 reducers:

```
[user@host ~]$ mardrerun -Dmapreduce.job.reduces=8 -i dataset.fastq
    -l 15
```

The following command shows a similar example but for paired-end execution, allowing two mismatches and using the default prefix length (20) and 8 reducers:

```
[user@host ~]$ mardrerun -Dmapreduce.job.reduces=8 -i dataset1.
   fastq -p dataset2.fastq -m 2
```

3.2 Configuration

MaRDRe can be configured by means of the *mardre.conf* file located at the *etc* directory. The available parameters are:

- mergeOutput (boolean). Allows to avoid merging the output files when is set to false. This also involves that the temporary intermediate files are not deleted (i.e., deleteTemp is forced to false). The default value is true.
- deleteTemp (boolean). Delete the intermediate files created by Hadoop. The default value is true.
- pairedJoin (boolean). Enable the use of a map-side join in paired-end scenarios. The default value is false, which means that the join is performed by executing a previous MapReduce job.
- compressTemp (boolean). Enable the compression of intermediate files in paired-end scenarios using the snappy codec. It requires the snappy library installed on the system and the Hadoop native library (i.e., libhadoop) compiled with snappy support. The default value is false.
- ioBufferSize (integer). The buffer size in bytes that is used for input read operations. It should probably be a multiple of the hardware page size (e.g., 4096). The default value is 65536 bytes.
- inMapperCombiner (boolean). Enable the use of the in-mapper combiner when possible. The default value is false.
- inMapperCombinerCacheSize (integer). Maximum number of entries cached by the in-mapper combiner. The default value is 32768 entries.

4 Compilation

In case you need to recompile the MaRDRe distribution, the prerequisites are:

1. Make sure you have Java Development Kit (JDK) version 1.7 or above
2. Make sure you have a working Apache Maven distribution version 3 or above:
   - https://maven.apache.org/install.html

In order to make the JAR distribution, just execute the following Maven command from within the MaRDRe root directory:

```
[user@host mardre]$ mvn package
```

The first time you execute this command, Maven will download all the plugins and related dependencies it needs to fulfill the command. From a clean installation of Maven, this can take quite a while. If you execute the command again, Maven will now have what it needs, so it will be able to execute the command much more quickly.

5 Contact

MaRDRe has been developed in the Computer Architecture Group [8] at the University of A Coruña [9] by the following authors:

- Roberto R. Expósito: http://gac.udc.es/~rreye
- Jorge Veiga: http://gac.udc.es/~jveiga
- Jorge González-Domínguez: http://gac.udc.es/~jgonzalezd
- Juan Touriño: http://gac.udc.es/~juan

To report any question, bug, requirement or information about MaRDRe, feel free to contact us at [1].

References

[1] MarDRe webpage. http://gac.udc.es/~rreye/mardre.

[2] Jorge González-Domínguez and Bertil Schmidt. ParDRe: faster parallel duplicated reads removal tool for sequencing studies. *Bioinformatics*, 32(10):1562–1564, 2016.

[3] ParDRe webpage. https://sourceforge.net/projects/pardre.

[4] Jeffrey Dean and Sanjay Ghemawat. MapReduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.

[5] Apache Hadoop. http://hadoop.apache.org.

[6] GNU General Public License version 3 (GPLv3). https://www.gnu.org/licenses/gpl-3.0.en.html.

[7] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The Hadoop distributed file system. In *Proceedings of the 26th IEEE Symposium on Mass Storage Systems and Technologies (MSST'10)*, pages 1–10, Incline Village, NV, USA, 2010.

[8] Computer Architecture Group. http://gac.udc.es/english.

[9] University of A Coruña. http://www.udc.gal/index.html?language=en.